Total Bandwidth Server

Moshiour Rahman Prince
Department of Electronic Engineering
Hamm-Lippstadt University of Applied Sciences
Lippstadt, Germany
moshiour-rahman.prince@stud.hshl.de

***Abstract*—Real-time systems must satisfy strict timing constraints to guarantee correct system behavior. While periodic task scheduling is well understood, the efficient handling of aperiodic (event-driven) tasks within the same system presents a fundamental challenge: aperiodic tasks must receive timely service without jeopardizing the schedulability of the periodic task set. The *Total Bandwidth Server* (TBS), introduced by Spuri and Buttazzo [1], addresses this problem elegantly within the Earliest Deadline First (EDF) framework by assigning synthetic deadlines to aperiodic requests in a manner that bounds their processor utilization. This paper presents the mathematical foundations of TBS, analyzes the scheduling algorithm and its runtime overhead, and demonstrates its application through a formal model built in UPPAAL. The model is evaluated on a representative workload of two periodic tasks and three aperiodic requests over a bounded observation window, and is formally verified for deadlock-freedom, schedulability (no missed deadline), and correctness of the TBS deadline-assignment rule, while the resulting response times and CPU utilization are extracted from the schedule. A second, relative-time model further establishes schedulability over unbounded time for a continuously arriving sporadic aperiodic stream. The results confirm that TBS provides an efficient, low-overhead mechanism for integrating aperiodic tasks into EDF-scheduled real-time systems while maintaining hard deadline guarantees for periodic tasks.**

***Index Terms*—real-time systems, aperiodic task scheduling, Total Bandwidth Server, Earliest Deadline First, UPPAAL, timed automata**

I. Introduction

A. Real-Time Systems

A real-time system is a computing system in which the correctness of a result depends not only on its logical correctness but also on the time at which it is produced [2], [3]. Missing a timing constraint in a hard real-time system can lead to catastrophic consequences, such as failures in automotive control, medical devices, or avionics. Soft real-time systems tolerate occasional deadline misses, though with degraded quality of service.

B. Periodic and Aperiodic Tasks

Tasks in real-time systems are typically classified by their activation pattern. A *periodic task* $\tau_i = (C_i, T_i)$ is activated regularly with period T_i and must complete its worst-case execution time (WCET) C_i before the deadline $d_{i,j} = r_{i,j}+T_i$, where $r_{i,j}$ is the arrival time of the j-th instance [4]. An *aperiodic task* arrives at unpredictable times with no known minimum interarrival time. Because aperiodic tasks cannot be characterized a priori, guaranteeing their deadlines is generally infeasible; instead, schedulers aim to minimize their response times while protecting the periodic task set [2].

C. Scheduling Challenges

The coexistence of periodic and aperiodic tasks creates a resource allocation dilemma. Naive approaches such as *background scheduling* service aperiodic tasks only during idle periods, leading to arbitrarily large response times under high periodic load [2]. Fixed bandwidth reservation methods improve predictability but fail to exploit spare processor capacity efficiently. A principled solution requires a mechanism that adapts dynamically to aperiodic workload while providing bounded, worst-case guarantees.

D. The Total Bandwidth Server

The Total Bandwidth Server (TBS), proposed by Spuri and Buttazzo [1], resolves this tension within the EDF scheduling framework. TBS reserves a fraction U_s of the processor for aperiodic tasks. When an aperiodic request arrives, TBS assigns it a synthetic deadline computed from the server utilization and the previous deadline, ensuring that the processor fraction allocated to aperiodic tasks never exceeds U_s. This approach is both bandwidth-optimal — it attains the best possible EDF schedulability bound (Section III-C) — and extremely lightweight at runtime, since deadline assignment requires only a single comparison and addition. It is worth noting that TBS is not optimal with respect to aperiodic *response times*; servers such as EDL and TB* improve responsiveness at the cost of higher complexity [12].

II. Background and Mathematical Fundamentals

A. Real-Time Scheduling

A schedulable task set requires that all tasks meet their deadlines under a given scheduling policy. Two foundational algorithms are Rate Monotonic (RM) scheduling [4], which assigns fixed priorities inversely proportional to periods, and Earliest Deadline First (EDF), a dynamic priority algorithm that always executes the task with the nearest absolute deadline.

B. Earliest Deadline First (EDF)

EDF is an optimal uniprocessor scheduling algorithm in the sense that, if any algorithm can schedule a given task set, then so can EDF [2]. A task set $\{\tau_1, \ldots, \tau_n\}$ is schedulable under

EDF if and only if the total processor utilization does not exceed one:

$$U_p = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1. \tag{1}$$

EDF is therefore more efficient than fixed-priority schemes such as RM, which can achieve at most $\ln 2 \approx 69.3\%$ utilization for arbitrary task sets [4].

C. Processor Utilization and Schedulability

When a Total Bandwidth Server with utilization U_s is introduced alongside a periodic task set with utilization U_p, the combined schedulability condition extends naturally from Eq. (1):

$$U_p + U_s \leq 1. \tag{2}$$

This result is proven formally in [1]: because TBS ensures that the CPU time allocated to aperiodic tasks never exceeds U_s in any interval, the periodic task set remains schedulable as if the server were itself a periodic task with utilization U_s.

D. Related Work: Aperiodic Servers

A number of *aperiodic servers* predate and surround TBS. Under fixed-priority (RM) scheduling, the Polling Server and the Deferrable Server [11] reserve a periodic budget for aperiodic requests, and the Sporadic Server [10] refines this with a budget-replenishment rule that preserves capacity until it is needed. These servers improve responsiveness but require a replenishment timer and additional bookkeeping. In the dynamic-priority (EDF) domain, Spuri and Buttazzo proposed a family of servers — including the Priority Exchange and EDL servers — that trade efficiency against implementation complexity [12]. TBS [1] occupies the lightweight extreme of this design space, reducing per-request work to a single deadline computation. The later Constant Bandwidth Server (CBS) [13] generalizes the bandwidth-reservation idea to soft and multimedia tasks with temporal isolation. The present paper focuses on TBS and uses these alternatives only as points of comparison.

III. Total Bandwidth Server Algorithm

A. Core Concept

TBS operates as a virtual server that sits alongside the periodic task set under EDF scheduling [1]. Rather than reserving a time slot or maintaining a replenishment queue, TBS works by assigning a synthetic absolute deadline to each incoming aperiodic request. This deadline positions the request in the EDF priority queue such that the server's long-run CPU consumption is bounded by its designated utilization U_s.

B. Mathematical Formulation

Let r_k denote the arrival time of the k-th aperiodic request, C_k its WCET, and d_{k-1} the deadline previously assigned to the $(k-1)$-th request. The TBS deadline assignment rule is [1]:

$$d_k = \max(r_k,\ d_{k-1}) + \frac{C_k}{U_s}, \tag{3}$$

with the initial condition $d_0 = 0$.

Variable definitions:

- d_k: Synthetic deadline assigned to the k-th aperiodic job.
- r_k: Arrival time (release time) of the k-th job.
- d_{k-1}: Deadline assigned to the previous aperiodic job.
- C_k: Worst-case execution time of the k-th job.
- U_s: Server utilization factor (bandwidth reservation), $0 < U_s < 1$.

The $\max$ operator in Eq. (3) prevents a new request from receiving an earlier deadline than the previous one, maintaining a non-decreasing deadline sequence that bounds cumulative CPU usage. Intuitively, each request "extends" the deadline window by C_k/U_s time units from the later of its own arrival or the previous deadline, ensuring that in any interval of length Δt, aperiodic processing does not exceed $U_s \cdot \Delta t$ [1].

C. Correctness: Theorem (Spuri and Buttazzo, 1996)

Given a set of n periodic tasks with total utilization U_p and a TBS with utilization U_s, the complete task set is schedulable under EDF if and only if $U_p + U_s \leq 1$ [1].

This is a significant result: it means TBS achieves the same schedulability condition as if the aperiodic workload were a perfectly shaped periodic task, with no additional analysis required.

D. Runtime Overhead and Complexity

The runtime overhead of TBS is minimal. At each aperiodic arrival, the server executes one comparison (the $\max$ operation) and one addition (Eq. (3)), giving a time complexity of $\mathcal{O}(1)$ per aperiodic request [1]. The only state maintained between arrivals is the single value d_{k-1}, making TBS memory-efficient. No replenishment timer, queue, or periodic interrupt is required, which distinguishes TBS from more complex servers such as the Sporadic Server [10] or Priority Exchange Server [12].

IV. Implementation and Application Example

A. UPPAAL Model Architecture

To formally verify the TBS properties and to reproduce its exact preemptive schedule, we model the system as a network of timed automata [8] in UPPAAL [6], [7]. The model is built from three reusable templates: PeriodicGenerator, AperiodicGenerator, and *Scheduler* (Appendix **??**). The first two release jobs, while the *Scheduler* plays the role of the single CPU and implements EDF, the one-unit execution step, and on-line deadline-miss detection. The size of the workload is given by the constants *N_PERIODIC* and *N_APERIODIC*; UPPAAL instantiates one generator process per task automatically from the corresponding identifier range, so the task set can be resized by editing just these two constants.

Time is modeled discretely. A single integer *currentTime* represents the current instant, and one clock *cpuClock* inside the *Scheduler* measures exactly one unit of CPU progress per cycle. The scheduler advances through a chain of *committed* locations *MissCheck*→*Release*→*Select*→*Dispatch* and then occupies either *Run* or *Idle* for one time unit before returning

to *MissCheck*. Because every parameter is an integer, re-evaluating the EDF decision at each integer instant reproduces the *exact* preemptive schedule that a continuous-time EDF scheduler would produce. A single broadcast channel *release* is pulsed once per instant by the scheduler, and all generators react to it, which keeps the whole network synchronized on one timeline.

1) Global Declarations: The periodic task set is held in three integer arrays giving the periods T_i, the worst-case execution times C_i, and the relative deadlines $D_i = T_i$, while the aperiodic workload is described by per-request arrival and computation arrays. The server bandwidth is kept as the exact fraction $U_s = \mathit{serverBandwidthNum}/\mathit{serverBandwidthDen} = 1/4$, so that $C_k/U_s = C_k \cdot \mathit{Den}/\mathit{Num}$ remains integer-valued. The remaining global state – the discrete clock *currentTime*, the index *runningJob* of the job on the CPU, the *deadlineMiss* flag, the observation *HORIZON*, and the broadcast channel *release* – is shared by all automata. The complete declaration block is given in Appendix **??**.

2) PeriodicGenerator: Each instance owns one periodic task *id* and consists of a single location *Waiting* with a self-loop. On every *release* pulse for which the period has elapsed, it releases a fresh job:

- Guard: *currentTime % periodicPeriod[id] == 0*.
- Sync: *release?*.
- Update *releasePeriodicJob(id)*: set the job's remaining time to *periodicWCET[id]*, its absolute deadline to *currentTime + periodicRelDeadline[id]*, and mark it active.

3) AperiodicGenerator: Each instance is one-shot: it moves from *Waiting* to *Released* at its arrival instant and assigns the TBS deadline.

- Guard: *currentTime == aperiodicArrival[id]*; Sync: *release?*.
- Update *assignTbsDeadline(id)*, which implements the deadline rule of Eq. (3) in the integer-preserving form $d_k = \max(r_k, d_{k-1}) + C_k \cdot \mathit{Den}/\mathit{Num}$, stores it in *aperiodicAbsDeadline[id]*, and marks the request active.

The routine keeps *lastAssignedDeadline* as the running server deadline d_{k-1} (initially $d_0 = 0$); its source is listed in Appendix **??**.

4) Scheduler (the CPU): The *Scheduler* drives time and runs EDF. Each cycle it (i) flags any job still unfinished at or past its deadline (*checkDeadlineMisses*), (ii) pulses *release!*, (iii) selects the earliest-deadline ready job, and (iv) executes that job, or idles, for one time unit.

- *selectEarliestDeadlineJob* scans every ready periodic and aperiodic job and returns the one with the smallest absolute deadline; deadline ties are broken in favor of the server (aperiodic), consistent with treating the synthetic deadline d_k as an ordinary EDF deadline.
- *executeOneTimeUnit* increments *currentTime*, decrements the running job's remaining time and, when an aperiodic job finishes, records its completion in *aperiodicFinish[k]* (its response time is finish − arrival).

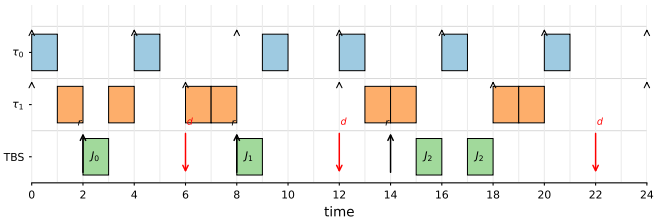

Fig. 1. Integrated EDF schedule of the two periodic tasks (τ_0, τ_1) and the three TBS-served aperiodic jobs (J_0, J_1, J_2). Black up-arrows mark periodic releases and aperiodic arrivals; red down-arrows mark the synthetic TBS deadlines d_0=6, d_1=12, d_2=22. Every deadline is met, and the CPU is idle for 6 of the 24 time units.

- *Dispatch* branches on *currentTime*: it enters *Run* when a job is ready, *Idle* when none is, and *Done* once *currentTime* reaches *HORIZON*.

The invariant *cpuClock ¡= 1* on *Run* and *Idle* forces exactly one unit of progress per cycle.

B. Application Example

The instantiated model uses two periodic tasks, $\tau_0 = (C_0{=}1, T_0{=}4)$ and $\tau_1 = (C_1{=}2, T_1{=}6)$, with utilizations $U_0 = 1/4$ and $U_1 = 1/3$, giving a periodic load of $U_p = 1/4 + 1/3 = 7/12$. The TBS reserves $U_s = 1/4$, so the combined utilization is

$$U_p + U_s = \tfrac{7}{12} + \tfrac{1}{4} = \tfrac{7}{12} + \tfrac{3}{12} = \tfrac{5}{6} \approx 0.83 \le 1,$$

which satisfies Eq. (2).

Three aperiodic requests arrive within the observation window: J_0 at $t = 2$ with $C_0 = 1$; J_1 at $t = 8$ with $C_1 = 1$; and J_2 at $t = 14$ with $C_2 = 2$. Applying Eq. (3) with $U_s = 1/4$ (so that $C_k/U_s = 4C_k$):

$$\begin{aligned} d_0 &= \max(2, 0) + 1/0.25 = 2 + 4 = 6, \\ d_1 &= \max(8, 6) + 1/0.25 = 8 + 4 = 12, \\ d_2 &= \max(14, 12) + 2/0.25 = 14 + 8 = 22. \end{aligned}$$

These synthetic deadlines are placed in the EDF ready queue alongside the periodic deadlines. Fig. 1 shows the resulting integrated execution timeline over $[0, 24)$, obtained directly from the deterministic model.

C. UPPAAL Verification Queries

Seven TCTL [9] queries, checked in the UPPAAL verifier, establish the key properties of the model; all of them hold for the workload above, and their exact formulas are collected in Appendix **??**. The first asserts that the network is deadlock-free, and the second is the formal schedulability check that no periodic or aperiodic deadline is ever missed. A third query confirms that the scheduler reaches the end of the observation window. Three further queries compare the assigned values in *aperiodicAbsDeadline* against the analytically derived deadlines 6, 12, and 22, validating that the integer implementation of Eq. (3) matches the theory. The final query confirms that even the last aperiodic request completes within the window.

D. Continuous, Unbounded Model

The model above is bounded to the window $[0, \mathit{HORIZON})$: once *currentTime* reaches the horizon the scheduler halts, so a run certifies only a finite prefix of the behavior. To obtain a guarantee that holds for *unbounded* time, we derive a second variant in which the periodic tasks are released indefinitely and the aperiodic workload is a *sporadic* stream whose inter-arrival time is chosen nondeterministically in $[MIT, MIT_{\max}]$. The horizon and the absolute clock are removed; instead, every temporal quantity — the periodic release timers, the job deadlines, and the server deadline d_{k-1} — is kept *relative* to the current instant and decremented by one each tick. Because EDF depends only on the ordering of deadlines and the TBS rule only on their differences, this relative encoding is exact, yet every integer stays bounded and the state space remains finite, so the temporal-logic queries still terminate; the core routines are listed in Appendix **??**. Boundedness additionally requires that the aperiodic stream be rate-limited to the reserved bandwidth, that is $MIT \geq C_k/U_s$; otherwise the assigned deadline $\max(r_k, d_{k-1}) + C_k/U_s$ would grow without bound. Under this model the nondeterministic *select* explores *every* admissible arrival pattern, and the verifier discharges $A\Box\, \neg$*deadlineMiss* (no deadline is ever missed) and $A\Box\, \neg$*deadlock*, together with a pool-overflow check, the bandwidth bound $A\Box\, (\mathit{serverSlack} \leq C_k/U_s)$, and a liveness property asserting that every aperiodic request is eventually served. The schedulability guarantee is thereby established over all time and all admissible arrival sequences, rather than on a single finite window.

V. Experimental Evaluation

The model is evaluated as a single integrated experiment that exercises both periodic and aperiodic load. The periodic set consists of $\tau_0 = (1, 4)$ and $\tau_1 = (2, 6)$, giving $U_p = 7/12 \approx 0.58$, and the server is configured with $U_s = 1/4$, so the schedulability bound $U_p + U_s = 5/6 \leq 1$ holds. Three aperiodic requests arrive at $t = 2, 8, 14$ with execution times $C = 1, 1, 2$. The system is run and verified over a window of $\mathit{HORIZON} = 24$ time units.

A. TBS Deadline Assignment

Table I lists the synthetic deadlines computed by the *assignTbsDeadline* routine. They reproduce exactly the values derived analytically from Eq. (3) and are confirmed in UPPAAL by the equality queries on *aperiodicAbsDeadline*.

TABLE I
Synthetic Deadlines Assigned by the TBS

Job	r_k	C_k	C_k/U_s	d_k
J_0	2	1	4	6
J_1	8	1	4	12
J_2	14	2	8	22

B. Integrated Schedule and Response Times

Running the deterministic model yields the timeline of Fig. 1. Table II reports the completion and response times of the three aperiodic jobs. Each response time is the completion instant recorded in *aperiodicFinish[k]* minus the arrival time r_k. All three requests finish well before their synthetic deadlines, and every periodic job also meets its deadline, so the *deadlineMiss* flag stays *false* throughout.

TABLE II
Aperiodic Response Times and Deadline Satisfaction

Job	Arrival	Finish	Resp.	d_k	Met?
J_0	2	3	1	6	yes
J_1	8	9	1	12	yes
J_2	14	18	4	22	yes

The two requests that arrive while the server is otherwise idle (J_0, J_1) are served immediately and complete in a single time unit. The last request J_2 has the largest response time (4 units): it is released while τ_1 is running and is then preempted once more by the periodic job of τ_0 at $t = 16$, both of which carry an earlier absolute deadline. Even so, it finishes at $t = 18$, four units ahead of its deadline at $t = 22$.

C. Utilization and Formal Verification

Over the window the CPU executes 18 of 24 units and idles for the remaining 6. The periodic tasks consume 14 units, matching $U_p = 14/24 \approx 0.58$ exactly, while the aperiodic share is $4/24 \approx 0.17$, comfortably below the reserved bandwidth $U_s = 0.25$; unused server capacity is left to the periodic set rather than wasted. All seven verification queries of Section IV-C are satisfied (Appendix **??**): the network is deadlock-free, no periodic or aperiodic deadline is ever missed, the scheduler reaches the end of the horizon, the assigned deadlines equal 6, 12, and 22, and the last request completes within the window.

D. Parameterization

Because the workload is driven entirely by the *N_PERIODIC*, *N_APERIODIC*, and bandwidth constants, heavier or lighter loads can be explored without changing the automata. Increasing *N_APERIODIC* (up to the array capacity *MAX_APERIODIC*) or raising U_s stresses the server, while the no-missed-deadline (schedulability) query remains the single criterion: as long as $U_p + U_s \leq 1$ and each C_k is honored, the query holds, which is exactly the guarantee of Theorem 1.

VI. Discussion

A. Advantages of TBS

The primary strength of TBS lies in its simplicity and optimality. The $\mathcal{O}(1)$ overhead per aperiodic arrival makes it suitable for resource-constrained embedded systems [1]. Because TBS operates within the EDF framework, it inherits EDF's optimality: any schedulable task set will be correctly scheduled. Unlike the Polling Server or Deferrable Server [11],

TBS does not waste bandwidth when no aperiodic requests are present, since unused capacity flows automatically to the periodic task set.

B. Limitations

TBS assumes that the WCET C_k of each aperiodic request is known at arrival time, which may be difficult to guarantee in practice [5]. Furthermore, TBS as originally defined handles only soft aperiodic tasks, since their deadlines are synthetic rather than application-specified. In distributed systems, additional mechanisms are needed to handle end-to-end deadlines and jitter constraints across nodes, as explored in [3], [5].

C. Interpretation of UPPAAL Results

The UPPAAL model allows formal verification of properties that are difficult to prove analytically for a specific task set instance. Verification of the deadlock-freedom query confirms that the automata composition is live, while the no-missed-deadline query provides evidence that the theoretical schedulability condition in Eq. (2) holds for the chosen parameters. Because the deadline-miss check is evaluated at every integer instant, a single passing run certifies the entire window rather than a sampled subset. For the bounded model this is still confined to one finite window; the continuous, relative-time variant of Section IV-D lifts it to an unbounded guarantee that holds for every admissible sporadic arrival sequence, leaving only the fixed periodic parameters as the scope of the claim. The equality queries on *aperiodicAbsDeadline* further tie the implementation back to the mathematics: they confirm that the integer-arithmetic form of the deadline rule used in the model coincides with the rational value $\max(r_k, d_{k-1}) + C_k/U_s$. The schedule itself illustrates the behavior of the server: aperiodic deadlines are serialized in non-decreasing order, and the periodic guarantees are never violated even when an aperiodic job wins a deadline tie and preempts a periodic task.

VII. Conclusion

This paper has presented the Total Bandwidth Server as an elegant and efficient solution to the challenge of integrating aperiodic tasks into EDF-scheduled real-time systems. The core deadline assignment rule (Eq. (3)) achieves bandwidth bounding with $\mathcal{O}(1)$ runtime overhead and a simple schedulability condition ($U_p + U_s \leq 1$). The formal UPPAAL model, built from three reusable templates and driven by a deterministic discrete-time scheduler, verifies these properties on a concrete task set of two periodic tasks and three aperiodic requests: it is deadlock-free, misses no deadline, and assigns the synthetic deadlines predicted by the theory, while keeping the aperiodic share within the reserved bandwidth. A second, relative-time model removes the observation horizon and drives the system with a continuously arriving sporadic aperiodic stream, lifting the no-missed-deadline result to an unbounded, all-patterns guarantee. Because the workload is parameterized by a few constants, the same model serves as a reusable test bench for other configurations. Future work could extend the model to distributed systems with precedence constraints, as motivated by [5], or compare TBS empirically against alternative aperiodic servers such as the Constant Bandwidth Server [13].

References

[1] M. Spuri and G. C. Buttazzo, "Scheduling Aperiodic Tasks in Dynamic Priority Systems," *Real-Time Systems*, vol. 10, no. 2, pp. 179–210, Dec. 1996.

[2] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York: Springer, 2011.

[3] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*, 2nd ed. New York: Springer, 2011.

[4] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[5] G. Fohler, T. Lennvall, and G. Buttazzo, "Improved Handling of Soft Aperiodic Tasks in Offline Scheduled Real-Time Systems using Total Bandwidth Server," in *Proc. 8th IEEE Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, Nice, France, Oct. 2001.

[6] A. David, K. G. Larsen, A. Legay, M. Mikučionis, and D. B. Poulsen, "UPPAAL SMC Tutorial," *International Journal on Software Tools for Technology Transfer*, vol. 17, no. 4, pp. 397–415, Jul. 2015.

[7] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a Nutshell," *International Journal on Software Tools for Technology Transfer*, vol. 1, no. 1–2, pp. 134–152, Dec. 1997.

[8] R. Alur and D. L. Dill, "A Theory of Timed Automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, Apr. 1994.

[9] R. Alur, C. Courcoubetis, and D. L. Dill, "Model-Checking in Dense Real-Time," *Information and Computation*, vol. 104, no. 1, pp. 2–34, May 1993.

[10] B. Sprunt, L. Sha, and J. P. Lehoczky, "Aperiodic Task Scheduling for Hard-Real-Time Systems," *Real-Time Systems*, vol. 1, no. 1, pp. 27–60, Jun. 1989.

[11] J. K. Strosnider, J. P. Lehoczky, and L. Sha, "The Deferrable Server Algorithm for Enhanced Aperiodic Responsiveness in Hard Real-Time Environments," *IEEE Transactions on Computers*, vol. 44, no. 1, pp. 73–91, Jan. 1995.

[12] M. Spuri and G. C. Buttazzo, "Efficient Aperiodic Service under Earliest Deadline Scheduling," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, San Juan, Puerto Rico, Dec. 1994, pp. 2–11.

[13] L. Abeni and G. Buttazzo, "Integrating Multimedia Applications in Hard Real-Time Systems," in *Proc. 19th IEEE Real-Time Systems Symposium (RTSS)*, Madrid, Spain, Dec. 1998, pp. 4–13.